03

Part 3 · Pipelines, Processing & DataOps

How data moves and changes: loading patterns, correctness guarantees, streaming, CDC and the tooling that ships it.

- ETL vs ELT · Pipelines
- Incremental loading · Watermarks
- Back-date refresh · Idempotency
- Exactly-once · Upsert
- Batch vs Streaming
- Lambda vs Kappa
- CDC · Delta Live Tables
- dbt · CI/CD · DataOps

3.1 Data Pipelines: ETL vs ELT

★★★

IN ONE LINE

Automated flows that extract data, transform it, and load it — ETL transforms before loading, ELT after.

A **data pipeline** is an automated, repeatable flow that moves and shapes data from sources to destinations. The most common design question is **where the transformation happens**.

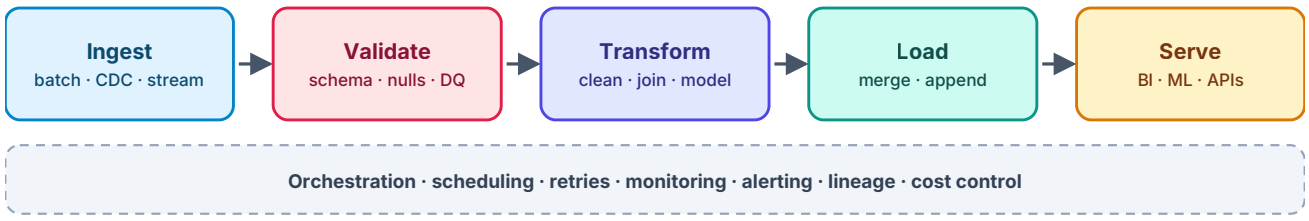


Figure 3.1a — Anatomy of a data pipeline

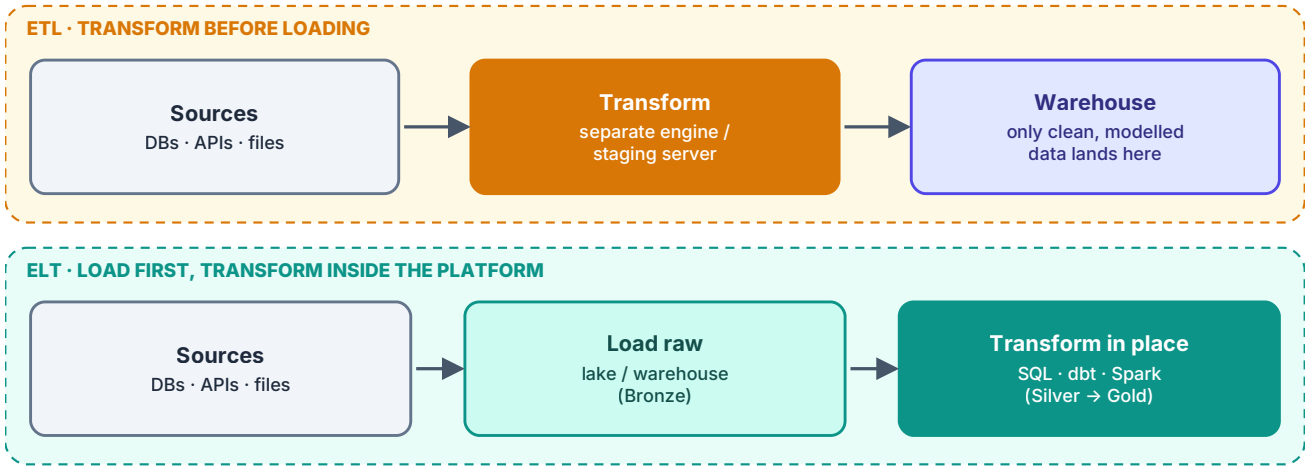


Figure 3.1b — ETL vs ELT

	ETL	ELT
Order	Extract → Transform → Load	Extract → Load → Transform
Where transform runs	Dedicated ETL tool / staging server	Inside the warehouse / lakehouse (SQL, dbt, Spark)
Raw data kept?	Usually not	**Yes** — reprocess any time
Fits	Strict compliance (mask before landing), legacy warehouses	Cloud warehouses / lakehouses with cheap, elastic compute
Flexibility	Rigid: schema decided up front	High: new models from the same raw data
Typical tools	Informatica, Talend, SSIS	Fivetran / Airbyte + dbt, Spark, Lakeflow

INTERVIEW ANGLE

Modern stacks are mostly **ELT** because storage and compute are cheap and decoupled. The honest nuance: *some* transformation (PII masking, light validation) often still happens before landing — "EtLT".

3.2 Incremental Data Loading

★★★

IN ONE LINE

Loading only the data that is new or changed since the previous run.

Instead of reloading everything every run (**full load**), an **incremental load** moves only new or changed data since the last run. It is faster, cheaper and the basis of almost every production pipeline.

SOURCE TABLE orders

id	amount	updated_at
1	500	10-17 09:00
2	300	10-17 18:30
3	750	10-18 08:10
4	120	10-18 11:45

green = changed after the watermark

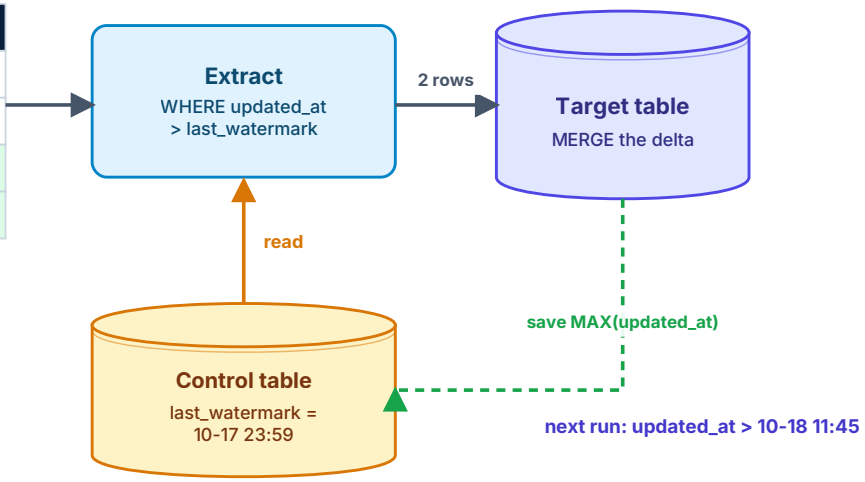


Figure 3.2 — High-water-mark incremental load

Pattern	How it works	Watch out for
Timestamp / watermark column	`WHERE updated_at > :last_ts`	Misses **hard deletes**; late commits; clock skew
Monotonic ID	`WHERE id > :last_id`	Only catches inserts, not updates
CDC (log-based)	Read inserts / updates / deletes from the DB log	More setup; ordering and initial snapshot
Partition / date based	Load yesterday's partition each day	Late-arriving data in old partitions
File based	Track which files were already ingested (Auto Loader, COPY INTO)	Files overwritten in place
Snapshot diff	Compare today's full snapshot with yesterday's	Expensive for large tables

SQL · INCREMENTAL PATTERN

```sql
-- 1) read the watermark   2) extract delta   3) merge   4) advance watermark
SELECT last_ts FROM etl_control WHERE table_name = 'orders';

MERGE INTO silver.orders t
USING (SELECT * FROM src.orders
       WHERE updated_at >= :last_ts - INTERVAL 1 HOUR) s   -- small overlap = safety margin
   ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

UPDATE etl_control SET last_ts = (SELECT MAX(updated_at) FROM src.orders) WHERE table_name = 'orders';
```

COMMON TRAP

Use a **small overlap** (`>=` last watermark minus a margin) and make the load **idempotent** (MERGE). Otherwise rows committed late, or sharing the exact boundary timestamp, are silently lost.

3.3 Watermarks ★★★

IN ONE LINE

A marker of progress: the last loaded value in batch, or the point in event time the stream considers complete.

"Watermark" has **two different meanings** in data engineering — say which one you mean.

1 · Batch high-water mark

The stored value (a timestamp or ID) of the **last record successfully loaded**. The next incremental run extracts everything beyond it. It lives in a control table (see 3.2).

2 · Streaming event-time watermark

A moving threshold that says **"we assume no more events older than T will arrive"**. It tells the engine when a window can be finalised and when old state can be discarded.



#	event time	max seen	watermark (max − 5 min)	result
1	10:02	10:02	09:57	accepted
2	10:07	10:07	10:02	accepted
3	10:13	10:13	10:08	accepted · window 10:00–10:05 closes
4	10:03	10:13	10:08	LATE (10:03 < 10:08) → dropped

Figure 3.3 — Event-time watermark with 5 minutes of allowed lateness

Watermark = (maximum event time seen so far) − (allowed lateness). Once the watermark passes a window's end, the window is finalised and results are emitted; events arriving later than that are **dropped** (or routed to a side output). A longer lateness tolerates more disorder but increases latency and state size — the core trade-off.

SPARK STRUCTURED STREAMING

```
from pyspark.sql import functions as F

result = (events
  .withWatermark('event_time', '5 minutes')           # tolerate 5 min of lateness
  .groupBy(F.window('event_time', '5 minutes'), 'device_id')
  .agg(F.count('*').alias('n')))
```

REMEMBER

Event time = when the thing happened (in the payload). **Processing time** = when the system sees it. Networks and mobile clients make them differ; watermarks exist to bridge that gap.

3.4 Back-date Refresh (Lookback & Backfill)

★★★

IN ONE LINE

Re-processing past periods to include late data, corrections or new logic.

Data is rarely complete when first processed: events arrive late, upstream systems correct history, and bugs or logic changes require rebuilding the past. **Back-dated refresh** (a lookback window and/or a **backfill**) means re-processing historical periods and safely overwriting the old results.



Figure 3.4 — Normal run vs lookback window vs backfill (each box is a daily partition)

Scenario	Strategy
Late-arriving facts (mobile events, delayed files)	Rolling **lookback window**: each run re-processes the last N days, not just today.
Upstream correction / restatement	Re-extract the corrected range; **MERGE** or overwrite the affected partitions.
Logic change or bug fix	**Backfill** the historic range in parallel date-chunks after deploying the new code.
Late-arriving dimension	Insert an "unknown" placeholder member; update facts once the real dimension row lands.

PYSPARK · PARTITION-SCOPED OVERWRITE

```
# Delta: atomically replace only the affected days (idempotent!)
(df.write.format('delta')
   .mode('overwrite')
   .option('replaceWhere', "event_date >= '2026-10-16' AND event_date <= '2026-10-19'")
   .save('/lake/silver/events'))
```

INTERVIEW ANGLE

Orchestrators parameterise the run date (Airflow `{{ ds }}`, `catchup=True`, `airflow dags backfill`). The prerequisite for all of this is **idempotency** — running the same date twice must give the same result (next topic). Also mention **bitemporal** thinking: event date vs load date lets you answer "what did we know on day X?"

3.5 Idempotency ★★★

IN ONE LINE
Running a pipeline step multiple times yields the same result as running it once.

An operation is **idempotent** if running it once or many times produces the **same final result**. Pipelines *will* be retried (task failure, scheduler restart, backfill), so every step must be safe to re-run.

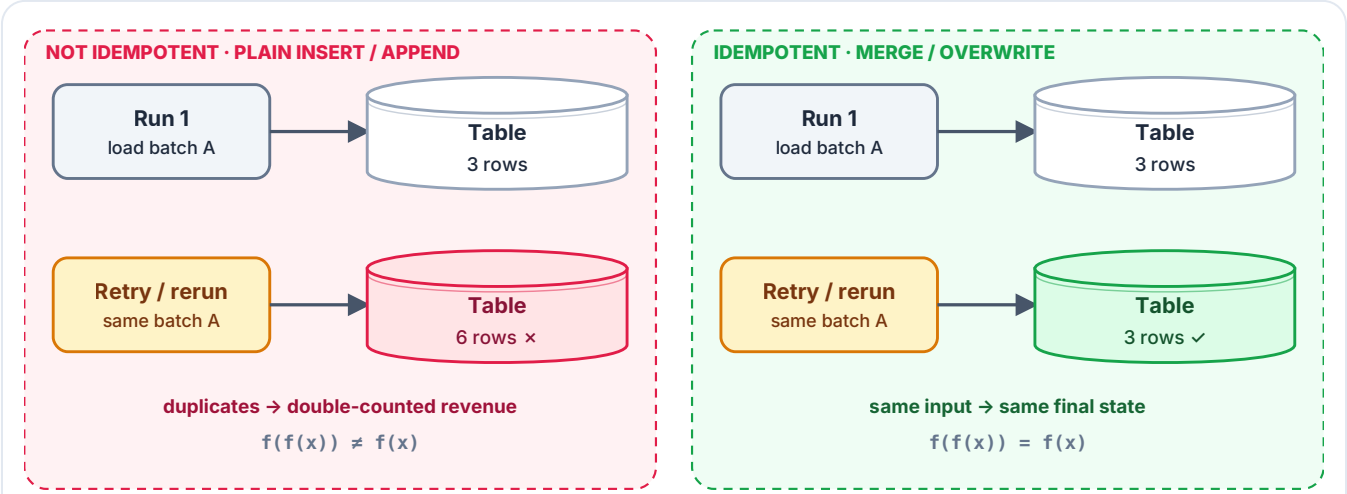


Figure 3.5 — Retrying a load: append duplicates, MERGE does not

Technique	How it makes a write idempotent
MERGE / upsert on a business key	Re-applying the same row updates it to the same values.
Overwrite the partition (`replaceWhere` , dynamic partition overwrite)	The run owns its slice: delete-then-write it atomically.
Deduplicate with a natural / deterministic key	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY ts DESC) = 1`
Track processed units	Record file names / offsets / batch IDs already applied (Auto Loader, checkpoints, `txnAppId`).
Deterministic transforms	No `now()` or `random()` inside the logic; derive IDs by hashing keys.
Write-then-swap	Build in a temp location, then atomic rename / table swap.

COMMON TRAP
`INSERT INTO ... SELECT` and `mode("append")` are the classic non-idempotent writes. Also watch hidden state: `current_timestamp()` or auto-increment keys generated during the load change on every retry.

INTERVIEW ANGLE
Say this sentence: **"I design each task so it can be re-run for any date range without side effects — usually by partition overwrite or MERGE on a stable key."**

3.6 Exactly-Once Processing

★★★

IN ONE LINE

Making sure each record's effect is applied once, despite retries and failures.

Messaging systems can promise three delivery semantics. **Exactly-once** is the hardest: in a distributed system with failures, a message can be sent once but processed twice (or vice versa). In practice it is achieved as **at-least-once delivery + idempotent (or transactional) processing** — the *effect* happens once even if the delivery repeats.

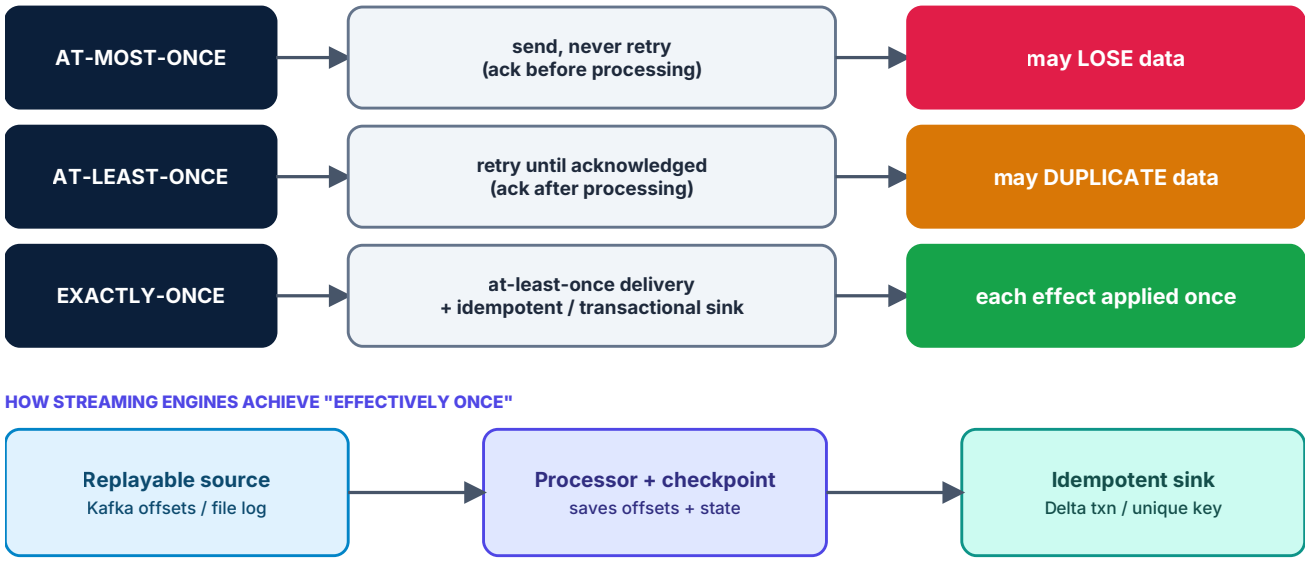


Figure 3.6 — Delivery semantics and the checkpoint-replay-dedupe recipe

Kafka

- **Idempotent producer** — broker de-duplicates retries per producer/partition.
- **Transactions** — atomic writes across partitions and offset commits (read-process-write).
- Consumers must read with `read_committed`.

Spark Structured Streaming + Delta

- **Checkpoint** stores source offsets and operator state.
- Replayable source + **idempotent Delta commits** (batch/txn IDs) = end-to-end exactly-once.
- `foreachBatch` sinks must be made idempotent by you (e.g. MERGE).

COMMON TRAP

"Exactly-once" only holds *within* the system that implements it. The moment you write to an external, non-idempotent sink (an email API, a payment call, a plain REST endpoint) you are back to at-least-once and must dedupe yourself with an idempotency key.

3.7 Upsert (MERGE)

★★★

IN ONE LINE

Update the row if the key exists, insert it if it does not — atomically.

Upsert = *update + insert*: if a row with the same key exists, update it; otherwise insert it. In SQL it is the `MERGE` statement. It is the workhorse behind incremental loads, CDC apply, SCD2 and idempotent pipelines.

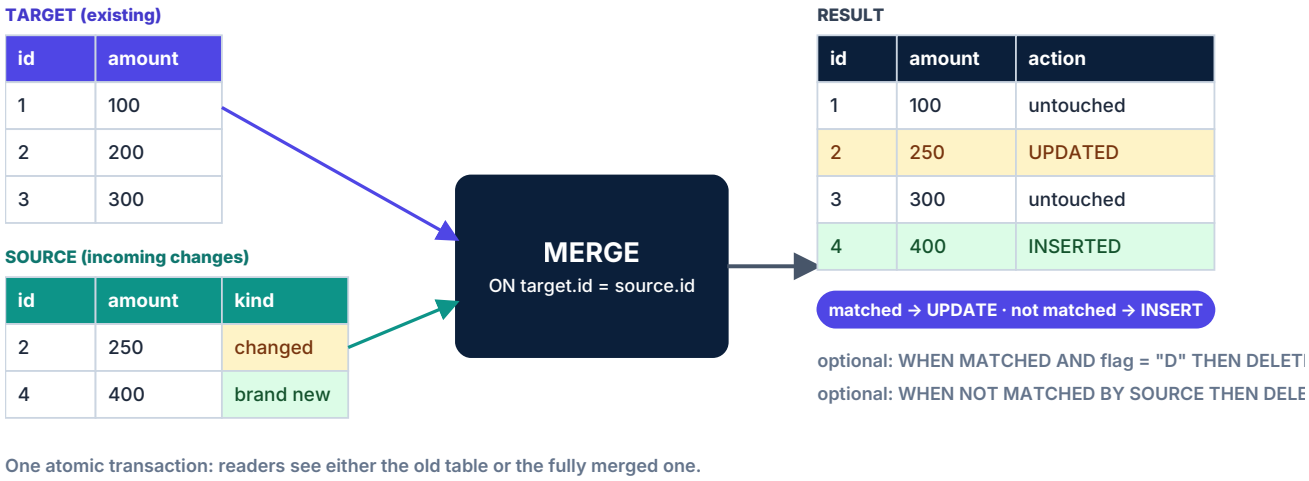


Figure 3.7 — MERGE: update matched rows, insert the rest

SQL · DELTA LAKE MERGE

```sql
MERGE INTO silver.customers AS t
USING updates AS s
   ON t.customer_id = s.customer_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Dialect names

- Delta / Snowflake / SQL Server / Oracle: `MERGE`
- PostgreSQL: `INSERT ... ON CONFLICT DO UPDATE`
- MySQL: `INSERT ... ON DUPLICATE KEY UPDATE`
- Mongo: `update` with `upsert: true`

Pitfalls

- **Duplicate keys in the source** → "multiple source rows matched" error or wrong results. De-duplicate first (latest per key).
- Unbounded `ON` clause scans the whole target — add partition / cluster filters.
- Merge rewrites files; many tiny merges cause small-file bloat.

INTERVIEW ANGLE

The standard CDC pattern is: **keep only the latest change per key from the batch, then MERGE**. Mention ordering by a sequence number / LSN, not by arrival time.

3.8 Batch vs Streaming Data Processing

★★★

IN ONE LINE

Batch handles bounded data on a schedule; streaming handles unbounded data continuously.

Batch processes a finite, bounded dataset on a schedule (hourly, nightly). **Streaming** processes an unbounded flow of events continuously, as they arrive. Many modern engines treat batch as a special case of streaming.

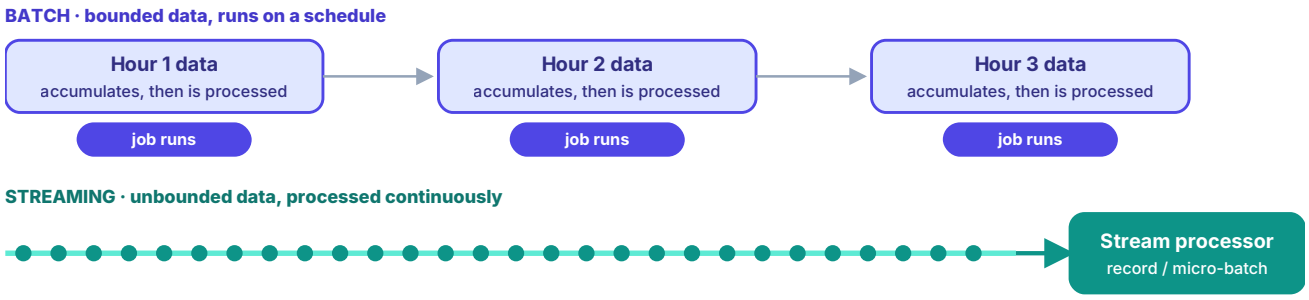


Figure 3.8a — Bounded vs unbounded processing

	Batch	Streaming
Latency	Minutes to hours	Milliseconds to seconds
Data	Bounded (a file, a day's partition)	Unbounded (events, clicks, sensor readings)
Complexity	Simpler to build, test, backfill	Harder: ordering, late data, state, exactly-once
Cost	Cheap — compute only while running	Always-on (or micro-batched) compute
Use when	Reports, ML training, nightly aggregates	Fraud detection, monitoring, real-time personalisation
Tools	Spark batch, dbt, SQL, Airflow	Kafka, Flink, Spark Structured Streaming, Kinesis

Windowing — turning an endless stream into aggregable chunks



Figure 3.8b — The three window types (dots are events)

Micro-batch vs continuous

Spark Structured Streaming defaults to **micro-batches** (latency ~100 ms–seconds); Flink is **record-at-a-time** (lower latency). A trigger like `availableNow` runs the streaming code as an incremental batch job — a cheap middle ground.

Correctness vocabulary

Event time vs **processing time**, **watermarks** for lateness, **stateful** operators, **checkpoints**, and **exactly-once** semantics.

INTERVIEW ANGLE

Do not say "streaming is always better". A mature answer: **pick the latency the business actually needs**; use batch (or `availableNow`) when hourly is fine, and streaming only where freshness has real value.

3.9 Lambda vs Kappa Architecture

★★★

IN ONE LINE

Two blueprints for serving both historical accuracy and real-time freshness.

Two classic patterns for combining **accuracy** (batch) with **freshness** (streaming).

Lambda architecture (Nathan Marz)

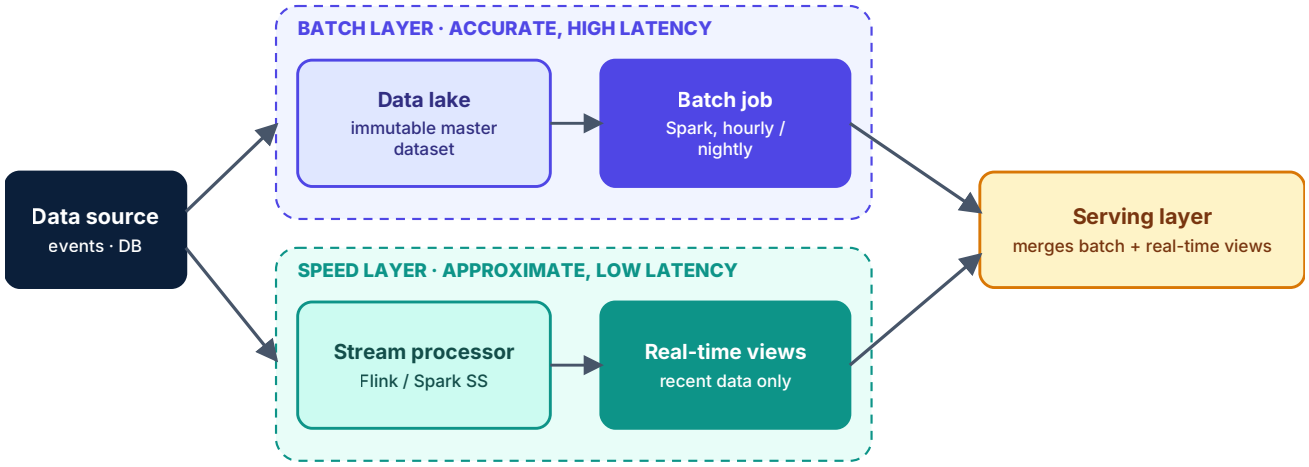


Figure 3.9a — Lambda: two parallel paths merged at serving time

Kappa architecture (Jay Kreps)

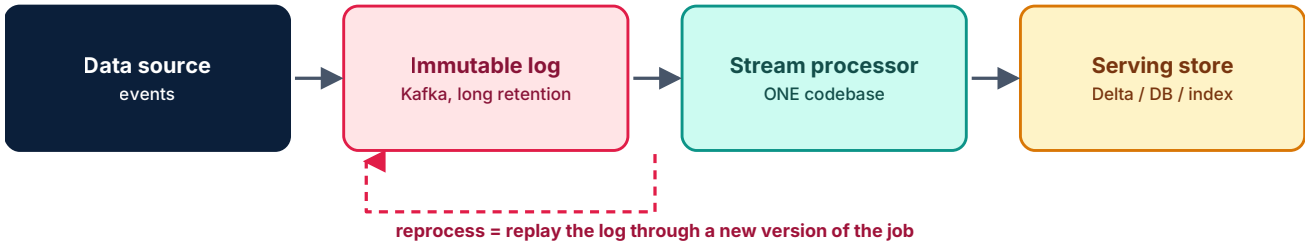


Figure 3.9b — Kappa: everything is a stream; the log is the source of truth

	Lambda	Kappa
Layers	Batch + speed + serving	Single streaming layer
Codebases	**Two** (must produce identical logic)	**One**
Reprocessing	Re-run batch job on the lake	Replay the log with new code
Pros	Accurate historical results; mature	Simple; one pipeline to maintain
Cons	Duplicate logic, drift, higher ops cost	Needs a long-retention replayable log; heavy streaming skills

INTERVIEW ANGLE

The modern answer: **lakehouse tables (Delta) unify both** — the same table is a streaming sink and a batch source with ACID, so you often get Kappa-like simplicity without keeping everything in Kafka. This is sometimes called the "Delta architecture" or "streaming-first lakehouse".

3.10 Change Data Capture (CDC)

★★★

IN ONE LINE

Capturing row-level inserts, updates and deletes from a source and propagating them downstream.

Change Data Capture identifies and delivers *row-level changes* (inserts, updates, **deletes**) from a source database to downstream systems, usually in near real time. Log-based CDC reads the same transaction log (WAL / binlog) the database uses for recovery — so it adds almost no load and captures every change in order.

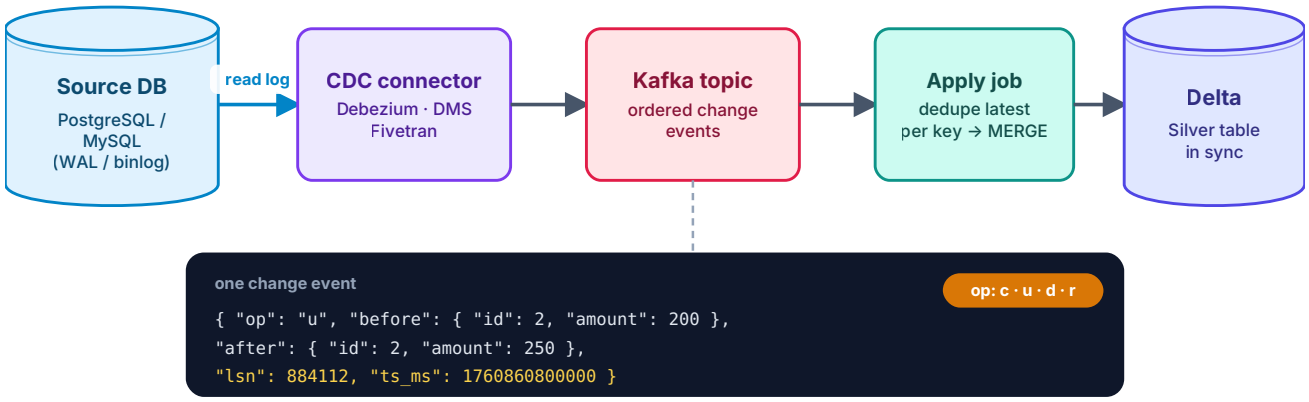


Figure 3.10 — Log-based CDC from an OLTP database into a Delta table

Method	How	Captures deletes?	Load on source
Log-based	Read WAL / binlog / redo log	✓ yes	Very low
Trigger-based	DB triggers write changes to an audit table	✓ yes	High (adds write overhead)
Query / timestamp	Poll `updated_at > last_ts`	× no (unless soft delete)	Medium (repeated queries)
Snapshot diff	Compare full extracts	✓ yes	High (full scans)

Implementation checklist

- **Initial snapshot** + then stream the log from a consistent position.
- Preserve **ordering** (LSN / sequence number) per key.
- Apply with **MERGE** after keeping the latest event per key.
- Handle **deletes** (hard delete or tombstone) and **schema changes**.
- Keep the raw change log in Bronze for replay.

Related tooling

- **Debezium** (open source, on Kafka Connect), AWS DMS, Fivetran, Airbyte.
- **Delta Change Data Feed** — row-level changes *between Delta table versions*, for downstream propagation.
- **Lakeflow** `APPLY CHANGES INTO` — declarative CDC apply with SCD1/SCD2 and out-of-order handling.

SQL · CHANGE DATA FEED

```sql
-- Delta Change Data Feed: what changed since version 5?
SELECT _change_type, * FROM table_changes('silver.customers', 5);
-- _change_type: insert | update_preimage | update_postimage | delete
```

INTERVIEW ANGLE

Tie it together: **WAL (1.4) → CDC (3.10) → MERGE (3.7) → SCD2 (1.11)**. This chain is a favourite “walk me through replicating a production DB to the lakehouse” question.

3.11 Delta Live Tables (Lakeflow Declarative Pipelines)

★★★

IN ONE LINE

A declarative framework: define tables and quality rules, and the engine manages dependencies, incremental runs and retries.

Delta Live Tables (DLT) is Databricks' **declarative** framework for building reliable batch and streaming pipelines: you declare the *tables you want* and the queries that define them, not the orchestration steps. Databricks has since renamed it **Lakeflow Spark Declarative Pipelines** (the engine was also contributed to Apache Spark as *Spark Declarative Pipelines*). Existing DLT code still runs unchanged.

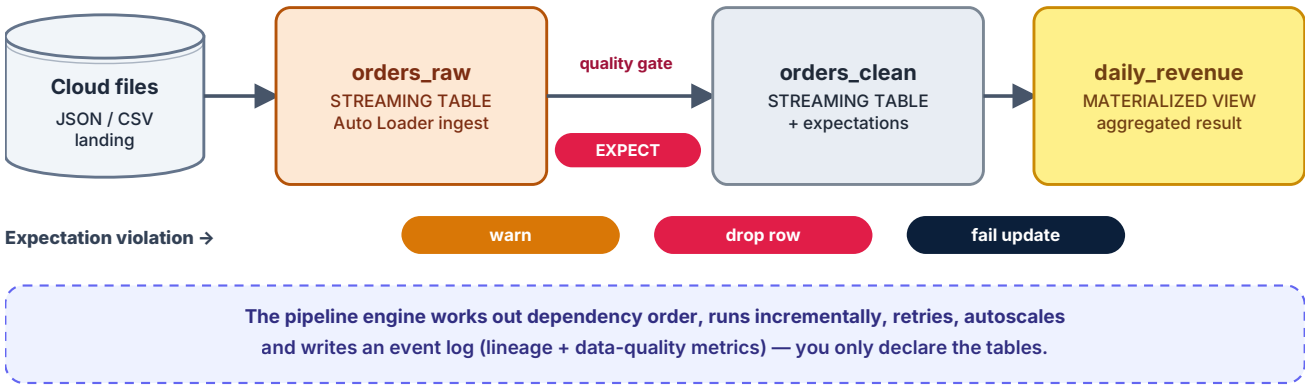


Figure 3.11 — A declarative pipeline with an expectation (quality gate) between Bronze and Silver

Object	What it is	Use for
Streaming table	Append-oriented table fed incrementally from a stream / file source	Ingestion (Bronze) and incremental Silver
Materialized view	Result of a query, kept up to date (incrementally where possible)	Aggregations and Gold tables
Temporary view	Intermediate result inside the pipeline only	Reusable logic, not published
Expectation	A data-quality rule: warn, drop row or fail the update	Enforce constraints without separate jobs

PYTHON · DECLARATIVE PIPELINE

```python
from pyspark import pipelines as dp          # (formerly: import dlt)
from pyspark.sql import functions as F

@dp.table                                        # streaming table
def orders_raw():
    return (spark.readStream.format('cloudFiles')
            .option('cloudFiles.format', 'json').load('/landing/orders'))

@dp.table
@dp.expect_or_drop('valid_id', 'order_id IS NOT NULL')
def orders_clean():
    return spark.readStream.table('orders_raw').withColumn('amount', F.col('amount').cast('double'))

@dp.materialized_view
def daily_revenue():
    return (spark.read.table('orders_clean')
            .groupBy(F.to_date('order_ts').alias('day')).agg(F.sum('amount').alias('revenue')))
```

SQL · SAME PIPELINE

```sql
CREATE OR REFRESH STREAMING TABLE orders_clean (
  CONSTRAINT valid_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW
) AS SELECT * FROM STREAM(orders_raw);

CREATE OR REFRESH MATERIALIZED VIEW daily_revenue AS
SELECT to_date(order_ts) AS day, SUM(amount) AS revenue FROM orders_clean GROUP BY 1;
```

Why teams use it

- Less boilerplate: dependencies and ordering are automatic.
- Built-in data quality via expectations.
- Incremental processing and retries handled for you.
- Built-in CDC: `APPLY CHANGES INTO` (SCD1 / SCD2).

Compare

- **vs hand-written Spark jobs + Airflow**: declarative, less glue code.
- **vs dbt**: dbt is SQL-first and warehouse-agnostic; Lakeflow pipelines are Databricks-native with streaming built in.

3.12 dbt (Data Build Tool)

★★★

IN ONE LINE

SQL-first transformation framework: modular models, tests, docs and lineage that run inside your warehouse.

dbt (data build tool) is the **T in ELT**: you write `SELECT` statements as version-controlled **models**, and dbt compiles, orders and runs them inside your warehouse / lakehouse, adding **tests**, **documentation** and **lineage**. It brings software-engineering practice (Git, CI, modularity) to analytics SQL.

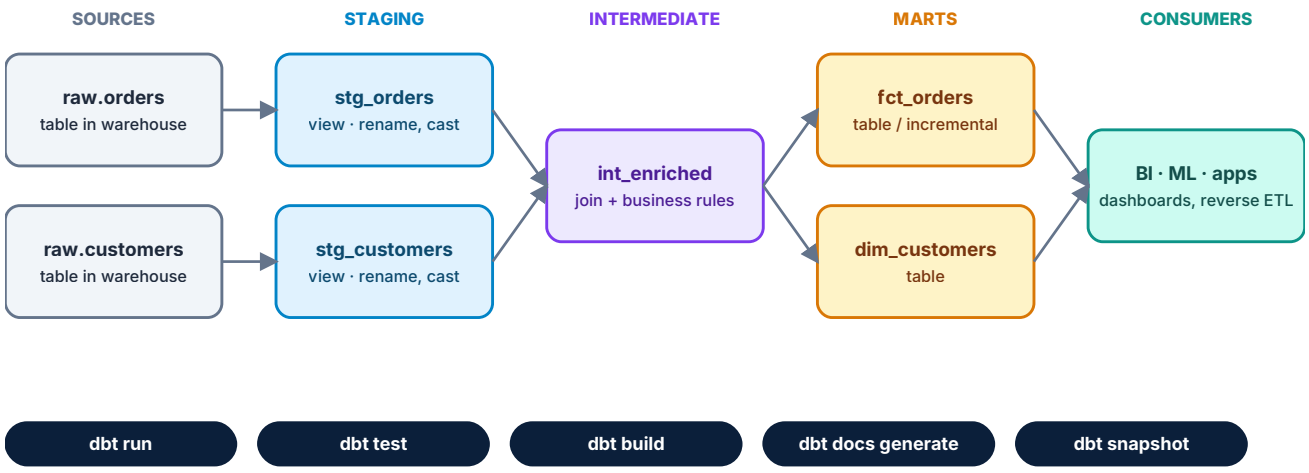


Figure 3.12 — A dbt project: sources → staging → intermediate → marts

DBT MODEL

```sql
-- models/marts/fct_orders.sql
{{ config(materialized='incremental',
unique_key='order_id') }}

select o.order_id, o.customer_id, o.amount,
o.updated_at
from {{ ref('int_orders_enriched') }} o
{% if is_incremental() %}
  where o.updated_at > (select max(updated_at)
from {{ this }})
{% endif %}
```

DBT TESTS

```yaml
# models/marts/schema.yml
models:
  - name: fct_orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: customer_id
        tests:
          - relationships:
              to: ref('dim_customers')
              field: customer_id
```

Materialization	What dbt creates	Use for
view	A SQL view	Light staging models (default)
table	Rebuilt table each run	Small/medium marts
incremental	Insert/merge only new or changed rows	Large facts
ephemeral	Inlined CTE, no object created	Reusable snippets
materialized_view	Warehouse-managed MV (where supported)	Auto-refreshed aggregates

Core concepts

- `ref()` / `source()` — build the dependency graph.
- **Tests**: generic (unique, not_null, accepted_values, relationships) and singular SQL tests.
- **Snapshots** — automatic SCD Type 2.
- **Macros** (Jinja), **seeds** (CSV), **docs** site with lineage.

What dbt is not

- Not an ingestion (E/L) tool.
- Not an orchestrator — it runs inside Airflow, Dagster, dbt Cloud or Workflows.
- It does not move data out of the warehouse; the engine does the compute.

3.13 CI/CD & DataOps

★★★

IN ONE LINE

Automating build, test and deployment of data pipelines — and running data like a reliable software product.

CI/CD applies software-delivery automation to data code: every change is built, tested and deployed through the same repeatable pipeline. **DataOps** is the broader practice that combines DevOps, agile and quality control to deliver reliable data *quickly and repeatedly*.

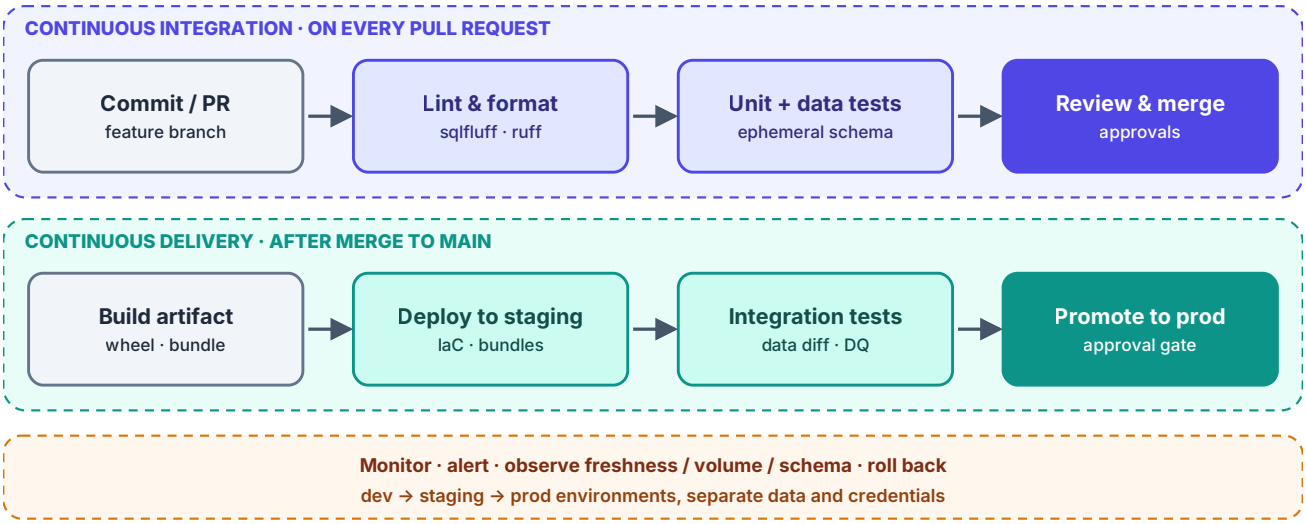


Figure 3.13 — A CI/CD pipeline for data (SQL, Spark, dbt, pipelines-as-code)

DataOps pillars

- **Version control** for code, configs, schemas, infrastructure.
- **Automated testing**: unit, integration, data-quality, regression.
- **Environments** (dev / staging / prod) with promotion gates.
- **Monitoring & observability** of freshness, volume, schema, cost.
- **Collaboration**, code review, shared definitions, data contracts.

Typical tools

- Git + GitHub Actions / GitLab CI / Azure DevOps / Jenkins
- Terraform, **Databricks Asset Bundles**, dbt Cloud / dbt slim CI (`state:modified+`)
- Great Expectations, Soda, dbt tests, Monte Carlo
- Airflow / Dagster / Workflows for orchestration

Test layer	Example	When
Static / lint	SQL style, type checks, secrets scan	Every commit
Unit	Transformation function on a tiny fixture DataFrame	Every PR
Data tests	unique / not_null / referential integrity on the built models	CI schema + after each prod run
Integration / regression	Run pipeline on a sample, compare output to the last release (data diff)	Before promotion
Monitoring	Freshness, volume anomalies, schema drift alerts	Continuously in prod

INTERVIEW ANGLE

Great interview detail: **test data, not just code**. Code can be correct while the data is wrong, so combine CI (code correctness) with runtime data-quality checks and observability. Also mention **idempotent deployments**, rollback plans and a CI schema per pull request so tests never touch production.